



IoT Based Weather and Air Quality Monitoring System using ESP32 and Telegram Alerts



Team Members

NAME	REG NO
Aryan Nair	24BDS0320
Prajith Senthil	24BAI0304
Sakrit Kafle	24BCE2952



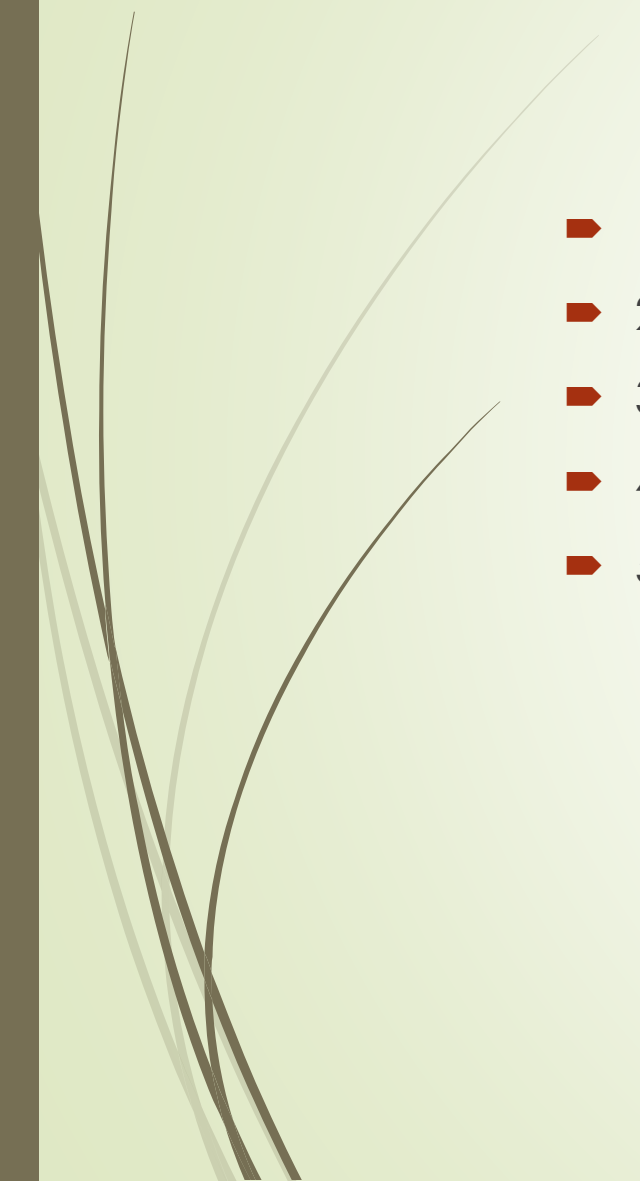
Aim (Objective) of the project

- The aim of this project is to design and implement an IoT-based environmental monitoring system using ESP32 that measures temperature, humidity, air pressure and gas levels.
- The system sends real-time alerts and data to a Telegram bot through WiFi, allowing users to monitor environmental conditions remotely.
- It also detects sudden spikes or dangerous gas levels and triggers warning alerts using LEDs and Telegram notifications.

Literature Survey			
S.No	Paper Title	Name of the Conference/Journal, Year	Technology Used
1	IoT Based Smart Air Quality Monitoring System	IEEE Sensors Journal, 2024	IoT, ESP32, MQ Gas Sensors
2	Real-Time Environmental Monitoring Using Wireless Sensor Networks	IEEE International Conference on IoT, 2023	Wireless Sensors, IoT, Cloud
3	Smart Weather Monitoring System Using IoT	International Journal of Engineering Research & Technology, 2024	Arduino/ESP32, DHT Sensors, WiFi
4	Air Pollution Monitoring Using IoT and Gas Sensors	Springer Environmental Monitoring Journal, 2023	MQ135 Sensor, Microcontroller, IoT
5	Design of IoT-Based Weather Station for Environmental Monitoring	IEEE Access, 2022	IoT, BMP Pressure Sensor, Microcontroller
6	Smart Environmental Monitoring System for Urban Areas	ACM International Conference on Smart Cities, 2024	IoT Sensors, Wireless Networks
7	Low-Cost IoT System for Air Quality Monitoring	Elsevier Environmental Technology Journal, 2023	ESP32, Gas Sensors, Cloud Monitoring
8	Real-Time Weather Monitoring Using Embedded Systems	International Conference on Embedded Systems, 2022	Embedded Systems, Sensors, WiFi
9	IoT-Based Smart Pollution	IEEE International Conference on Smart	IoT, Gas Sensors, Mobile Alerts



Websites referred

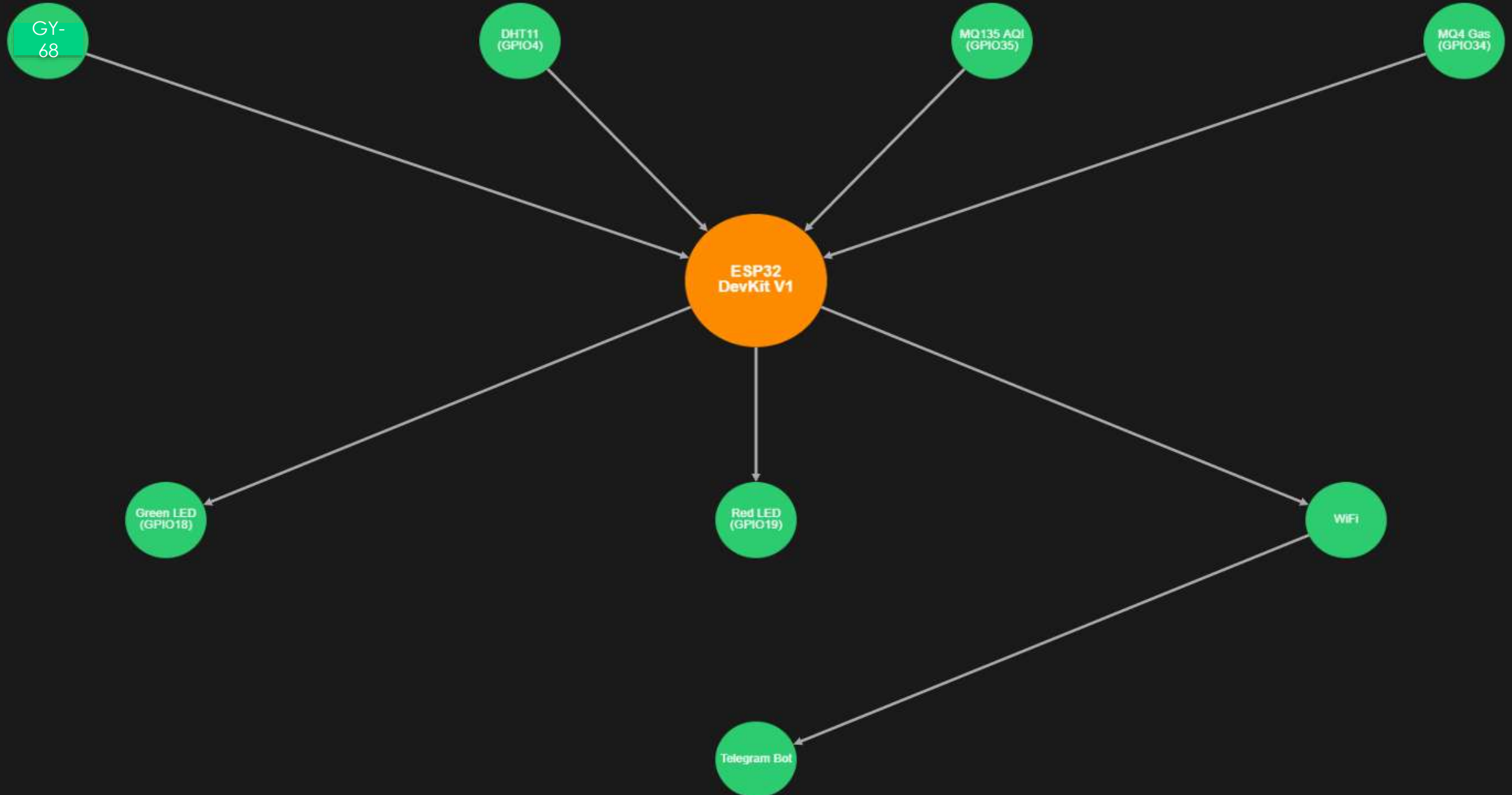
- 1. <https://randomnerdtutorials.com>
 - 2. <https://arduino.cc>
 - 3. <https://github.com>
 - 4. <https://iotdesignpro.com>
 - 5. <https://lastminuteengineers.com>
- 



Methodolgy/Block Diagram

- 
- 1. ESP32 connects to WiFi network.
 - 2. Sensors collect environmental data:
 - DHT11 – Temperature & Humidity
 - BMP085 – Pressure & Altitude
 - MQ4 – Methane Gas
 - MQ135 – Air Quality
 - 3. ESP32 processes sensor values and checks thresholds.
 - 4. If abnormal values or sudden spikes are detected:
 - Red LED turns ON
 - Telegram alert is sent.
 - 5. User can request live data using Telegram command (/data).

ESP32 Weather & Gas Monitoring System





Hardware components

- Hardware Components:
- ESP32 DevKit V1
- MQ4 Gas Sensor
- MQ135 Air Quality Sensor
- DHT11 Temperature & Humidity Sensor
- GY-68 Pressure Sensor
- Red and Green LEDs
- Jumper Wires
- Breadboard



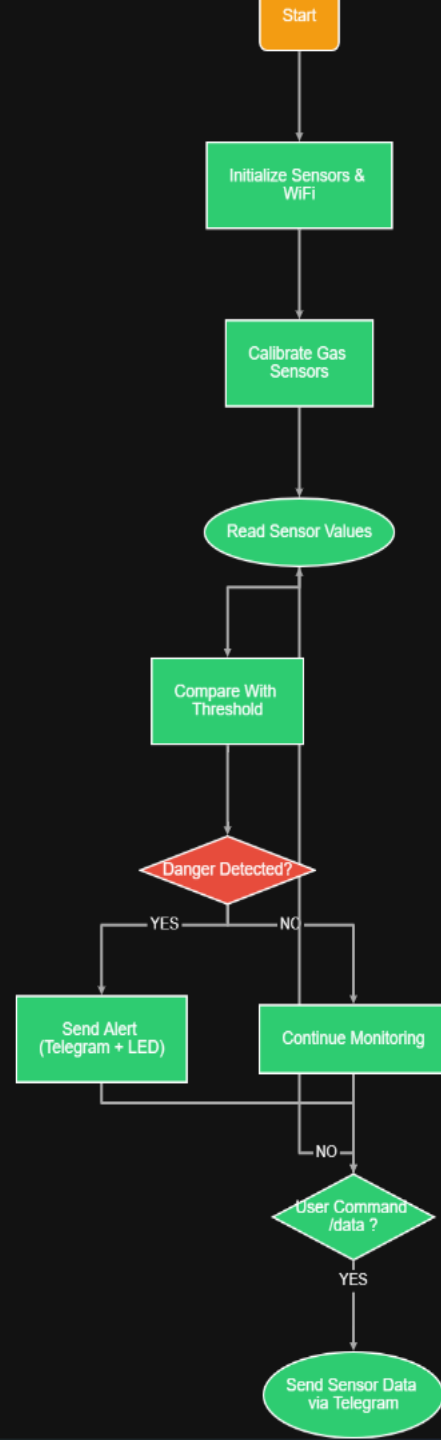
Software Details + algorithm/Alp flow chart

- Software Tools:

- - Arduino IDE
 - ESP32 Board Package
 - Telegram Bot API
 - WiFi Library
 - DHT Sensor Library
 - Adafruit BMP085 Library

- Algorithm:

- 1. Initialize sensors and WiFi connection.
 2. Calibrate gas sensors.
 3. Read sensor values periodically.
 4. Compare readings with threshold values.
 5. Detect sudden changes in temperature, pressure or gas.
 6. Send alerts through Telegram.
 7. Respond to user command /data to send current readings.





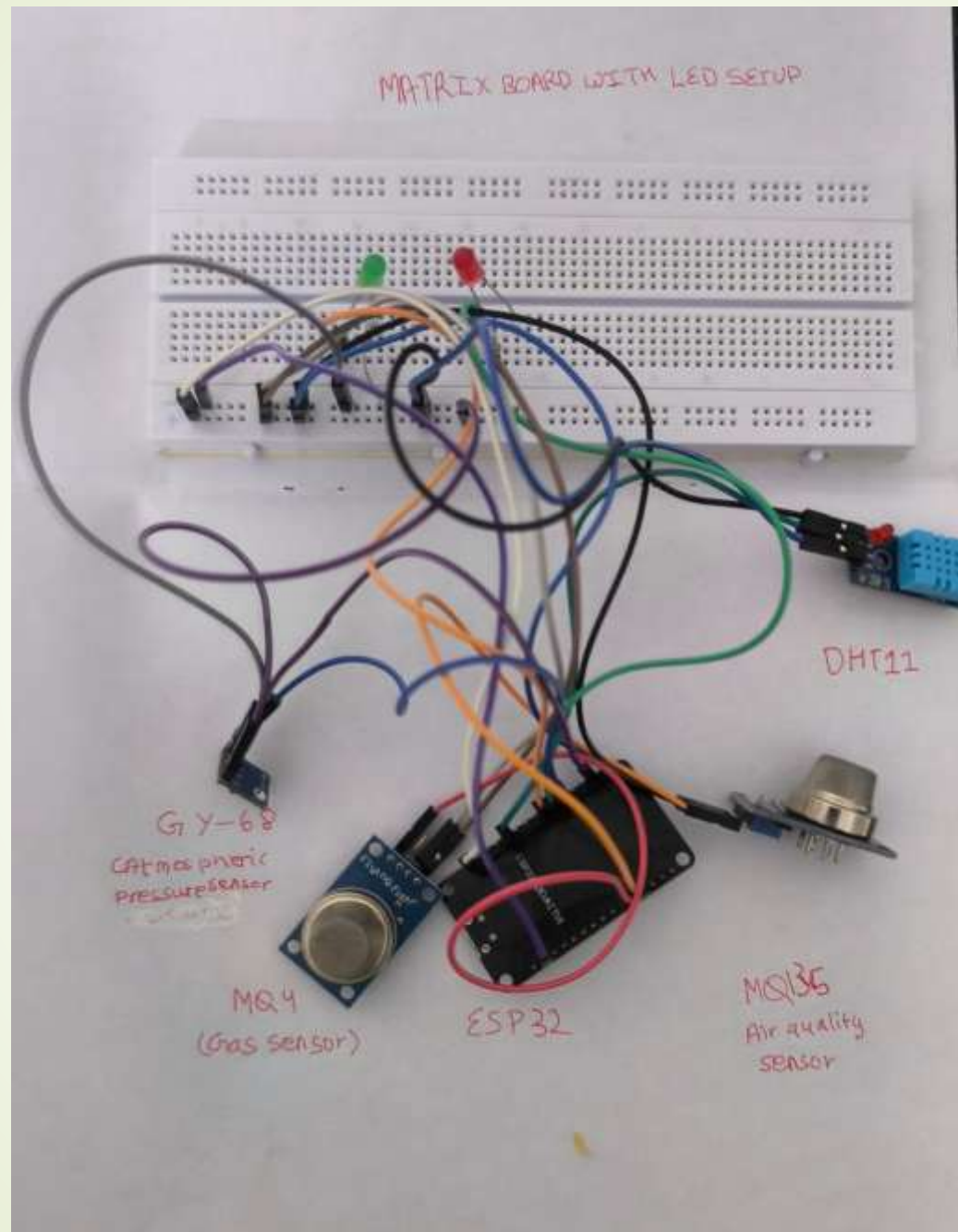
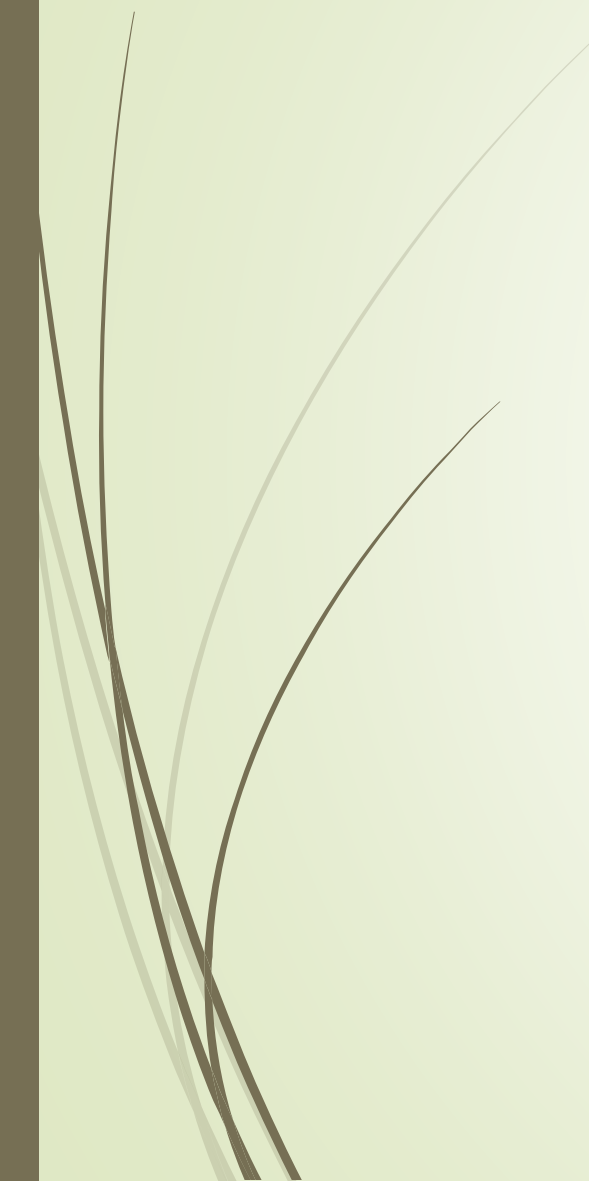
Work completed so far

- ESP32 hardware setup completed.
- Sensors successfully interfaced.
- WiFi connectivity established.
- Telegram bot integrated.
- Real-time sensor readings implemented.
- LED alert system working.
- Sudden spike detection algorithm implemented.



Images of the code and results

- ▀ Images include:
 - ESP32 circuit setup
 - Code implementation in Arduino IDE
 - Telegram bot alert messages




```
main.ino | Arduino IDE 2.3.8
File Edit Sketch Tools Help

main.ino
1 #include <WiFi.h>
2 #include <WiFiClientSecure.h>
3 #include <UniversalTelegramBot.h>
4 #include <ArduinoJson.h>
5 #include <Wire.h>
6 #include <Adafruit_BMP285.h>
7 #include <DHT.h>
8
9 // ===== WIFI =====
10 const char* ssid = "HOMER 20H Pro"; // CHANGE THIS
11 const char* password = "prajithrockz"; // CHANGE THIS
12
13 // ===== TELEGRAM =====
14 #define BOT_TOKEN "67342107384:AAEvvtANGtliclyAKS1qWpZqZw9hZw7W" // GET A NEW ONE
15 #define CHAT_ID "8626400926"
16
17 WiFiClientSecure client;
18 UniversalTelegramBot bot(BOT_TOKEN, client);
19
20 // ===== SENSORS =====
21 #define MQ4_PIN 34
22 #define MQ135_PIN 35
23 #define RED_LED 19
24 #define GREEN_LED 18
25
26 #define DHTPIN 4
27 #define DHTTYPE DHT11
28
```

```
main.ino | Arduino IDE 2.3.8
File Edit Sketch Tools Help

main.ino
29 DHT dht(DHTPIN, DHTTYPE);
30 Adafruit_BMP285 bmp;
31
32 int mq4Base = 0;
33 int mq135Base = 0;
34
35 // Shared data
36 float tempDHT, humidity, pressure, altitude, tempBMP;
37 int mq4Avg, mq135Avg;
38
39 // Tracking Previous Data for "Sudden Changes"
40 float prevTemp = 0.0, prevPressure = 0.0;
41 int prevMq4 = 0, prevMq135 = 0;
42
43 // Limits for Static Alerts
44 const float LIMIT_TEMP = 35.0;
45 const float LIMIT_PRESSURE = 1025.0;
46 const int LIMIT_GAS = 80;
47
48 // Limits for Sudden Changes (Geltas)
49 const float DELTA_TEMP = 1.0;
50 const float DELTA_PRESSURE = 4.0;
51 const int DELTA_GAS = 150;
52
53 // Flags and Timers
54 volatile bool alertGas = false;
55 volatile bool alertTemp = false;
56 volatile bool alertPressure = false;
57
```

```
volatile bool spikeGas = false;
volatile bool spikeTemp = false;
volatile bool spikePressure = false;

volatile bool isReady = false;

unsigned long lastAlertTime = 0;
const unsigned long ALERT_COOLDOWN = 300000; // 5 minutes

// ===== DATA TRANSLATION FUNCTIONS =====

String getGasStatus(int current, int base) {
  int difference = current - base;
  if (difference < 40) return "Normal 🟢";
  if (difference < LIMIT_GAS) return "Elevated 🟡";
  return "DANGER 🔴";
}

String getPressureStatus(float press) {
  if (press < 970.0) return "Low (Stormy) 🌪️";
  if (press > 1010.0) return "High (Clear) 🌤️";
  return "Normal (Fair) 🌤️";
}

// ===== SENSOR TASK (CORE 1) =====
void sensorTask(void * parameter) {
  for (;;) {
    int mq4Sum = 0;
```

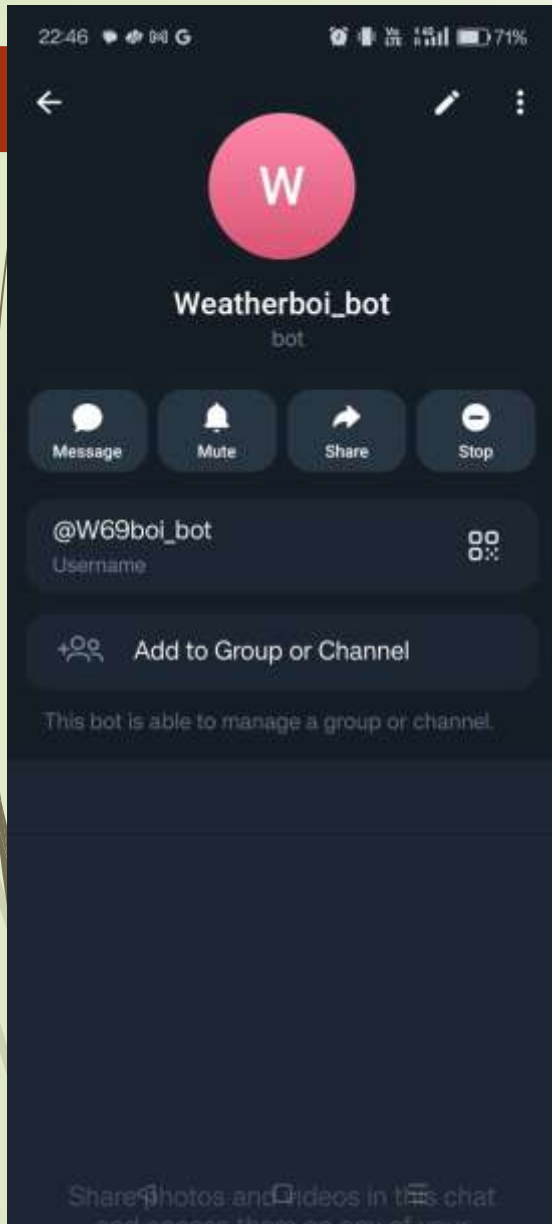
```
84 // ===== SENSOR TASK (CORE 1) =====
85 void sensorTask(void * parameter) {
86   for (;;) {
87     int mq4Sum = 0;
88     int mq135Sum = 0;
89
90     for (int i = 0; i < 10; i++) {
91       mq4Sum += analogRead(MQ4_PIN);
92       mq135Sum += analogRead(MQ135_PIN);
93       vTaskDelay(50 / portTICK_PERIOD_MS);
94     }
95
96     mq4Avg = mq4Sum / 10;
97     mq135Avg = mq135Sum / 10;
98
99     humidity = dht.readHumidity();
100     tempDHT = dht.readTemperature();
101     pressure = bmp.readPressure() / 100.0;
102     altitude = bmp.readAltitude();
103     tempBMP = bmp.readTemperature();
104
105     // 1. Check for Static Limits
106     alertGas = ((mq4Avg - mq4Base > LIMIT_GAS) || (mq135Avg - mq135Base > LIMIT_GAS));
107     alertTemp = (tempDHT > LIMIT_TEMP);
108     alertPressure = (pressure > LIMIT_PRESSURE);
```

```
main.ino | Arduino IDE 2.3.8
File Edit Sketch Tools Help

main.ino
109 alertTemp = (tempDHT > LIMIT_TEMP);
110 alertPressure = (pressure > LIMIT_PRESSURE);
111
112 // 2. Check for Sudden Spikes
113 if (abs(tempDHT - prevTemp) >= DELTA_TEMP) spikeTemp = true;
114 if (abs(pressure - prevPressure) >= DELTA_PRESSURE) spikePressure = true;
115 if (abs(mq4Avg - prevMq4) >= DELTA_GAS || abs(mq135Avg - prevMq135) >= DELTA_GAS) spikeGas = true;
116
117 // 3. Update Previous Variables for the next loop
118 prevTemp = tempDHT;
119 prevPressure = pressure;
120 prevMq4 = mq4Avg;
121 prevMq135 = mq135Avg;
122
123 // Update LEDs
124 if (alertGas || alertTemp || alertPressure || spikeGas || spikeTemp || spikePressure) {
125   digitalWrite(RED_LED, HIGH);
126   digitalWrite(GREEN_LED, LOW);
127 } else {
128   digitalWrite(RED_LED, LOW);
129   digitalWrite(GREEN_LED, HIGH);
130 }
131
132 Serial.println("Sensor task running");
133 vTaskDelay(2000 / portTICK_PERIOD_MS);
134 }
135
```

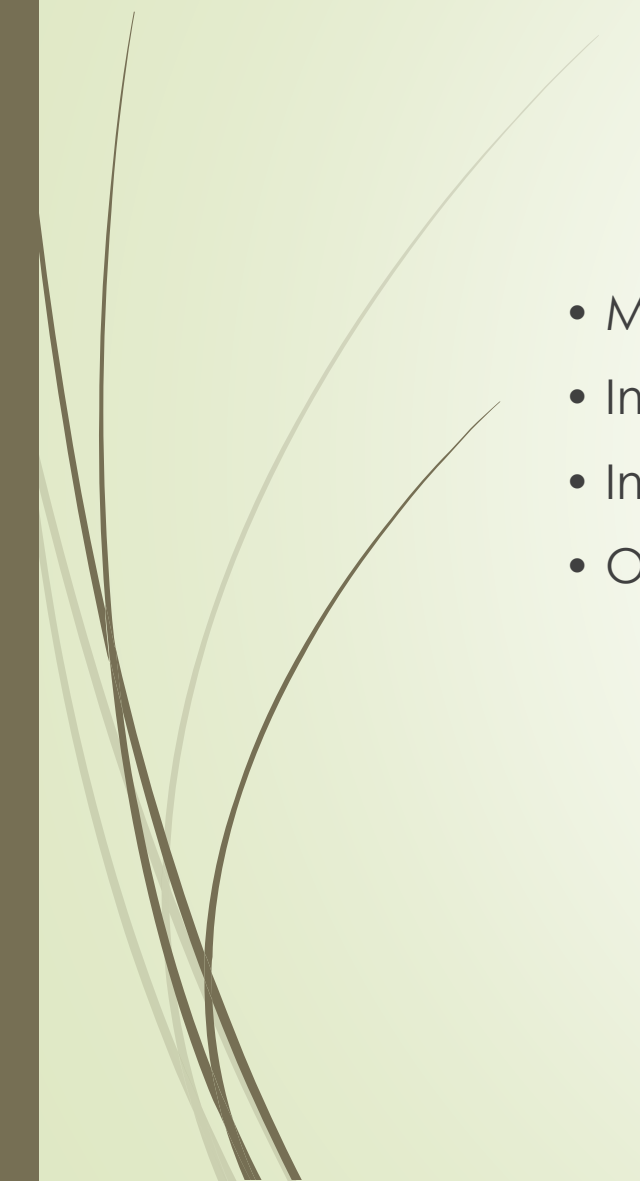
```
main.ino | Arduino IDE 2.3.8
File Edit Sketch Tools Help

main.ino
136 void telegramTask(void * parameter) {
137   for (;;) {
138     // 1. Proactive Alert Check (Static Limits - 5 Min Cooldown)
139     if (isReady && (alertGas || alertTemp || alertPressure) && (millis() - lastAlertTime > ALERT_COOLDOWN)) {
140       String alertMsg = "⚠️ WARNING: LIMIT EXCEEDED ⚠️\n\n";
141       if (alertGas) alertMsg += "🔴 High Gas/AQI Detected!\n";
142       if (alertTemp) alertMsg += "🔴 High Temp: " + String(tempDHT) + " °C\n";
143       if (alertPressure) alertMsg += "🔴 High Pressure: " + String(pressure) + " hPa\n";
144       bot.sendMessage(CHAT_ID, alertMsg, "");
145       lastAlertTime = millis();
146     }
147
148     // 2. Sudden Change Alert Check (triggers immediately)
149     if (isReady && (spikeGas || spikeTemp || spikePressure)) {
150       String spikeMsg = "⚡ URGENT: SUDDEN SPIKE DETECTED ⚡\n\n";
151       if (spikeGas) spikeMsg += "🔴 Rapid increase in Gas/AQI!\n";
152       if (spikeTemp) spikeMsg += "🔴 Temperature jumped rapidly!\n";
153       if (spikePressure) spikeMsg += "🔴 Sudden change in air pressure!\n";
154       spikeMsg += "📡 Send /data for current readings.";
155       bot.sendMessage(CHAT_ID, spikeMsg, "");
156
157       spikeGas = false;
158       spikeTemp = false;
159       spikePressure = false;
160     }
161   }
162 }
```





Work to be completed (after review-2)

- Make led lights functional
 - Improve accuracy of gas sensor calibration.
 - Implement data logging to cloud server.
 - Optimize power consumption of system.
- 



Cost analysis(for hardware experiments)

- Estimated Cost:
- ESP32 DevKit V1 – ₹450
- MQ4 Gas Sensor – ₹200
- MQ135 Sensor – ₹200
- DHT11 Sensor – ₹120
- BMP085 Sensor – ₹180
- LEDs + Resistors – ₹30
- Jumper wires & Breadboard – ₹150
- Total Estimated Cost ≈ ₹1330



Abstract

- Air pollution and sudden environmental changes create dangerous threats which can harm human health and safety. The project demonstrates an IoT-based system which monitors air quality and weather conditions through an ESP32 Devkit V1. The system uses DHT11 Temperature and Humidity Sensor and MQ-4 Gas Sensor and MQ-135 Gas Sensor and BMP085 Barometric Pressure Sensor to collect environmental data about temperature and humidity and air pressure and gas concentration.

The ESP32 processes sensor data to identify both normal conditions and sudden changes that surpass established limit points. The system sends alerts together with live data to users through a Telegram bot which operates using Wi-Fi connections. The system delivers an affordable solution which enables users to monitor environmental conditions in real time and receive remote alerts.

Plagiarism report for Abstract

Plagiarism Checker

Check Grammar

AI Detector

Pro

Scan Properties

Sources Found0

Words126

Characters852

View More Details

Plagiarism0%

0% Exact Match

0% Partial Match

Unique100%

Air pollution and sudden environmental changes create dangerous threats which can harm human health and safety. The project demonstrates an IoT-based system which monitors air quality and weather conditions through an ESP32 DevKit V1. The system uses DHT11 Temperature and Humidity Sensor and MQ-4 Gas Sensor and MQ-135 Gas Sensor and BMP085 Barometric Pressure Sensor to collect environmental data about temperature and humidity and air pressure and gas concentration. The ESP32 processes sensor data to identify both normal conditions and sudden changes that surpass established limit points. The system sends alerts together with live data to users through a Telegram bot which operates using WiFi connections. The system delivers an affordable solution which enables users to monitor environmental conditions in real time and receive remote alerts.



Congratulations!

No Plagiarism Found

Plagiarism Checker

Check Grammar

AI Detector

Pro

English

American

Copy

Download

Air pollution and sudden environmental changes create dangerous threats which can harm human health and safety. The project demonstrates an IoT-based system which monitors air quality and weather conditions through an ESP32 DevKit V1. The system uses DHT11 Temperature and Humidity Sensor and MQ-4 Gas Sensor and MQ-135 Gas Sensor and BMP085 Barometric Pressure Sensor to collect environmental data about temperature and humidity and air pressure and gas concentration. The ESP32 processes sensor data to identify both normal conditions and sudden changes that surpass established limit points. The system sends alerts together with live data to users through a Telegram bot which operates using WiFi connections. The system delivers an affordable solution which enables users to monitor environmental conditions in real time and receive remote alerts.



Impressive Content Quality

All your writing is flawless